Introduction to Embedded Systems

Course Code: 244066

Chapter 2: STM32 Microcontrollers

By

Dr. Muhammad Imran Nadeem

Zhengzhou University

Contents

1. **Overview**
   - Intro to STM32
   - ARM & Cortex Overview
2. Core Architecture & Execution
   - CPU Modes, Switching, Stack Usage
   - Instruction Set
   - Memory Map
3. System Resources
   - SysTick Timer
   - Interrupts & NVIC
   - CMSIS
4. STM32-Specific Features
   - Clock System (RCC)
   - DMA
5. Software & Tools
   - Driver Libraries
   - Development Workflow
6. Summary

2.1. Overview

ST Microelectronics

- Merge of SG Microelectronica and Thomson Semiconductors
- Founded in 1987
- ICs, smart cards, microcontrollers.



2.1. Overview

Key Features

- ARM 32-bit Cortex™-M3 core with embedded Flash and SRAM
  - Up to 512KB Flash memory
  - Up to 64KB SRAM
- Clock, reset and supply management
- Multiple communication peripherals
  - I2C, USART, SPI
- Sleep, Stop and Standby low power mode
- Multiple 16-bit timer
- Real-time clock
- DMA controller
- Easy development



Figure: STM32 Microcontroller.

2.1. Overview

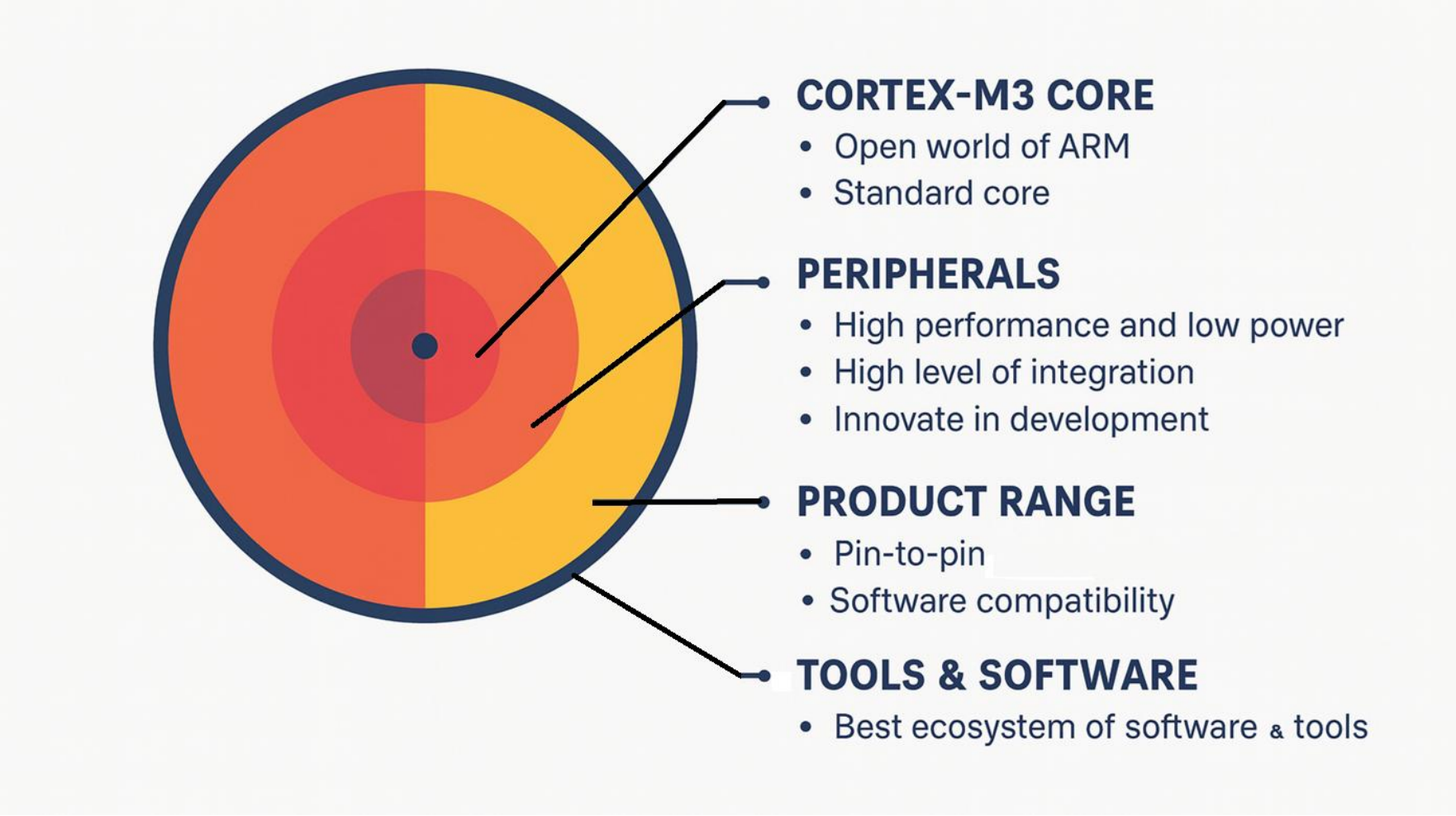
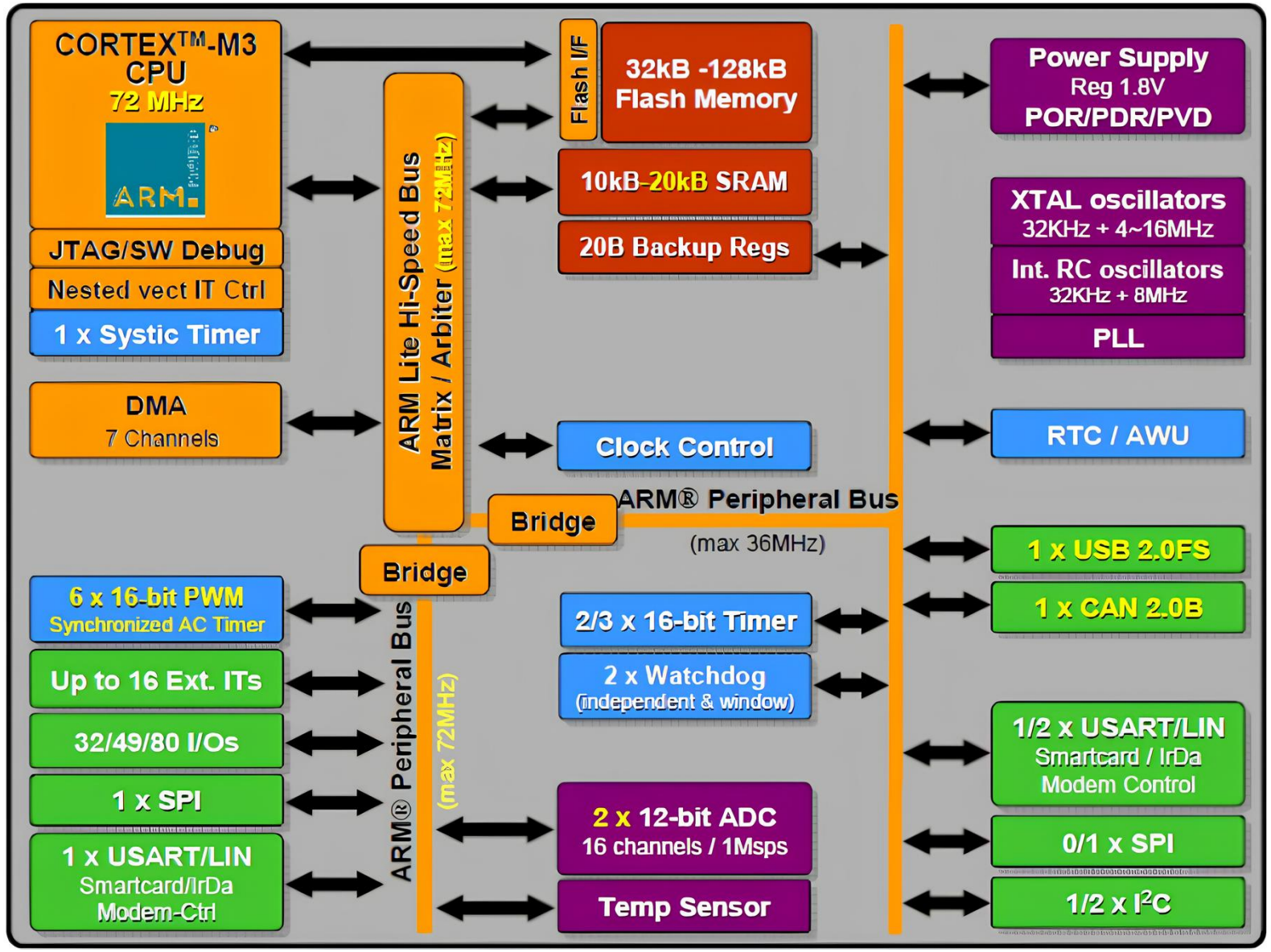


Figure: Core Components of STM32.

2.1. Overview

Block Diagram of STM32

- Multiple bus access
- Efficient DMA and rapid data flow
- Increased peripheral speed for better performance



2.1. Overview

STM32 Applications

- **Industrial**:
  - PLC
  - Inverters
  - Printers, scanners
  - Industrial networking
- **Building and security:**
  - Alarm systems
  - Video intercom
  - HVAC
- **Low power:**
  - Glucose meters
  - Power meters
  - Battery operated application
- **Appliances:**
  - Motor drive
  - Application control
- **Consumer:**
  - PC peripherals, gaming
  - Digital camera, GPS platforms
- **AI and Machine Learning**
  - Signal processing
  - Computer vision
  - Natural language processing

2.1. Overview

Advanced RISC Machine (ARM)

- Founded in 1990
- CPU IPs, physical Ips, compilers, SoC infrastructure
- ARM Dominates 75% of embedded market



2.1. Overview



Figure: Customer Base of ARM.

2.1. Overview

What is **Cortex ?**

- Next generation ARM Processor
- Consists of:

  Classic CPU(s)

  System peripherals
- 3 series:

  A series: for complex applications and OS

  R series: for real-time systems

  M series: optimized for cost/power sensitive systems



2.1. Overview

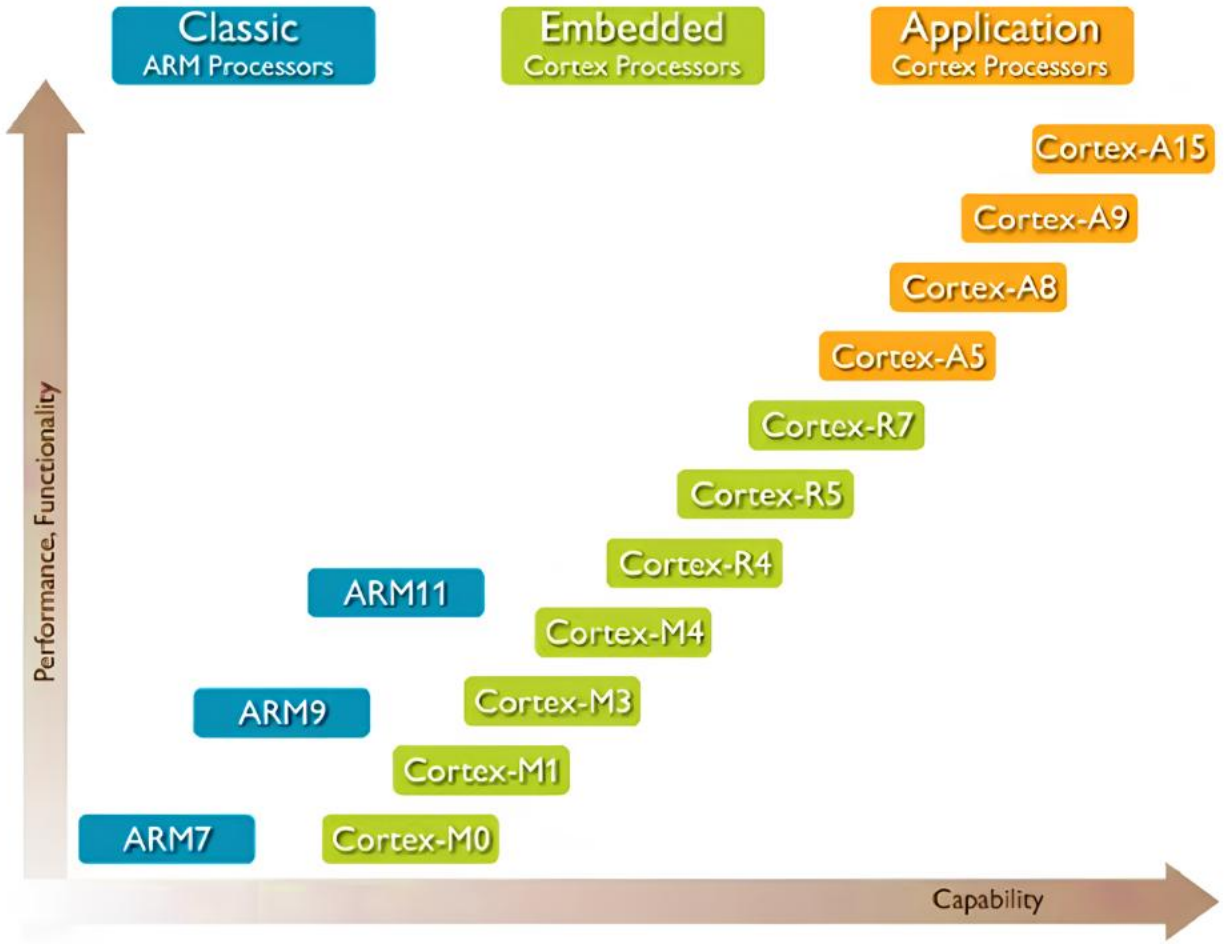


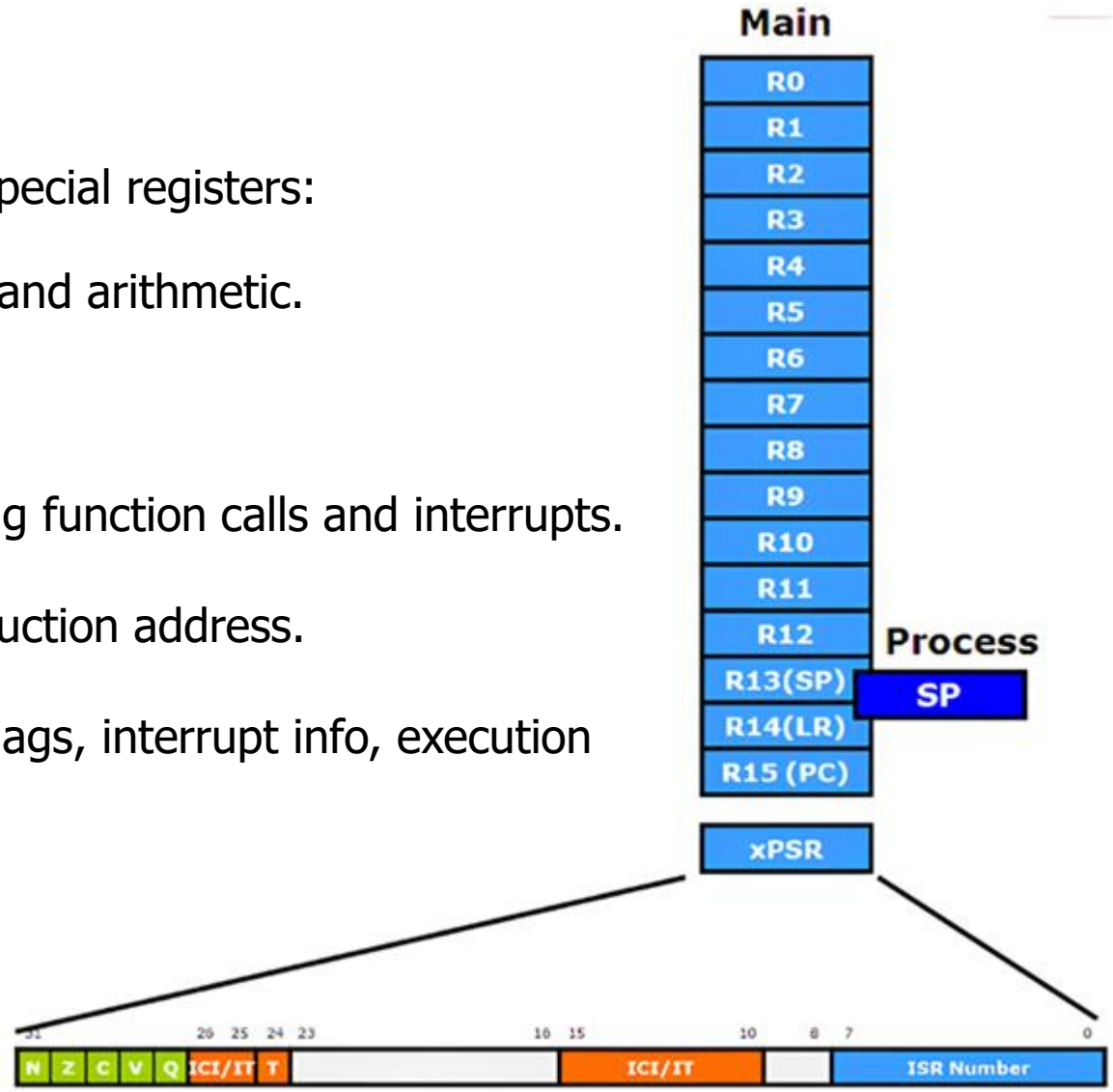
Figure: Classification of ARM.

2.1. Overview

CPU Core Registers

The Cortex-M has **16 general-purpose registers** plus special registers:

- **R0 – R12**: General-purpose registers for data storage and arithmetic.
- **R13 (SP)**: **Stack Pointer**
- **R14 (LR, Link Register)**: Stores return address during function calls and interrupts.
- **R15 (PC, Program Counter)**: Holds the current instruction address.
- **xPSR (Program Status Register)**: Holds condition flags, interrupt info, execution state.



Contents

1. Overview
   - Intro to STM32 (Applications, Block Diagram, Features)
   - ARM & Cortex Overview
2. **Core Architecture & Execution**
   - CPU Modes, Switching, Stack Usage
   - Instruction Set (Thumb-2)
   - Memory Map
3. System Resources
   - SysTick Timer
   - Interrupts & NVIC
   - CMSIS
4. STM32-Specific Features
   - Clock System (RCC)
   - DMA
5. Software & Tools
   - Driver Libraries
   - Development Workflow
6. Summary

2.2. Core Architecture & Execution

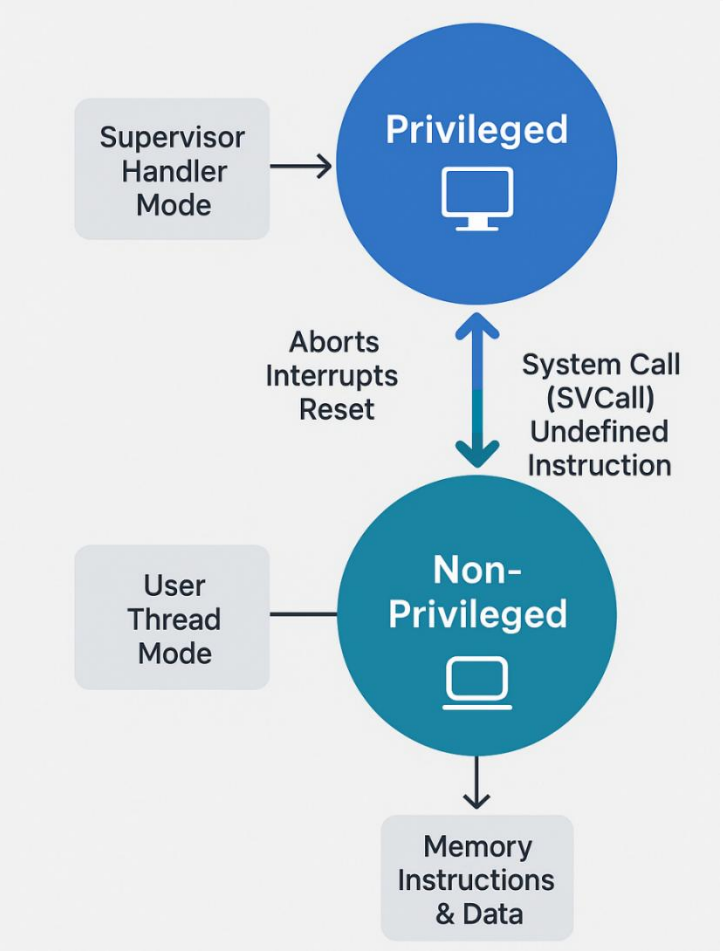
CPU Operation Modes
Two Main CPU Operation Modes:

Privileged Mode

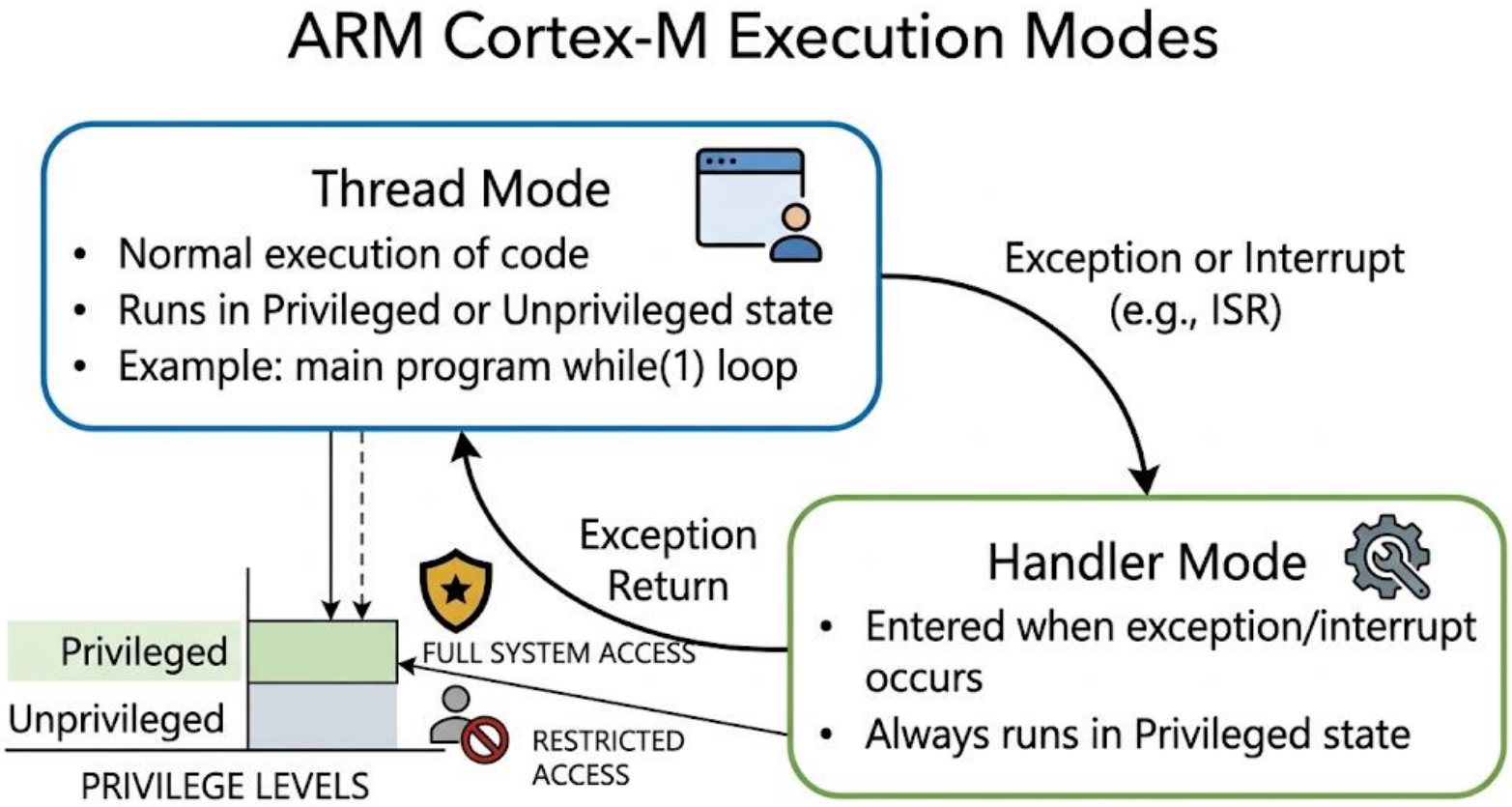
- Full access to all instructions, registers, and system resources.
- Can configure interrupts, access protected memory regions, and execute OS kernel code.

Non-Privileged Mode (User Mode)

- Limited access, cannot perform system-critical operations.
- Prevents user applications from accidentally (or maliciously) harming the system.



2.2. Core Architecture & Execution



2.2. Core Architecture & Execution

Stack Usage

ARM Cortex-M has two stack pointers:

- **Main Stack Pointer (MSP)** → Used by OS kernel, system exceptions, interrupts.
- **Process Stack Pointer (PSP)** → Used by user applications (in thread mode).

Out of reset, **MSP is active**. The OS can later configure **PSP** for user applications.

2.2. Core Architecture & Execution

What is an Interrupt?

An interrupt is a hardware or software event that tells the CPU to pause its current task and execute a special function called an **Interrupt Service Routine (ISR)**.

Purpose:

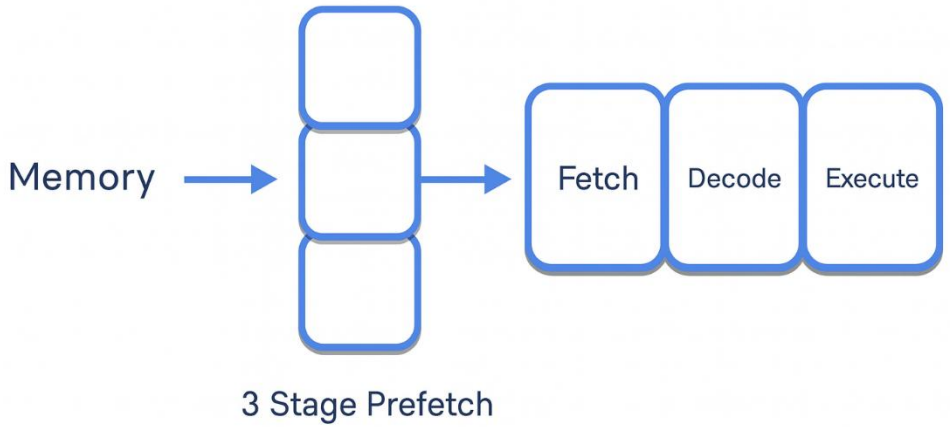
They allow the system to respond quickly to real-time events (like a button press, incoming data, or a timer overflow).

2.1. Overview

Instruction Pipeline

Cortex-M uses a 3-stage pipeline:

- **Fetch** → Instruction fetched from memory.
- **Decode** → Instruction is interpreted.
- **Execute** → Instruction is carried out.



This enables **parallelism**: while one instruction executes, the next is decoded, and the third is fetched → improving speed.

2.2. Core Architecture & Execution

Instruction Set

- The collection of **machine instructions** a CPU can understand and execute
- Defines the **language between software and hardware**

Parts of an Instruction

- **Opcode** → Operation to perform (e.g., ADD, MOV)
- **Operands** → Data or location to use (register, memory)

Example

```
ADD R0, R1, R2 → Adds R1 + R2, stores result in R0.
```

2.2. Core Architecture & Execution

ARM cores originally had **two instruction sets**:

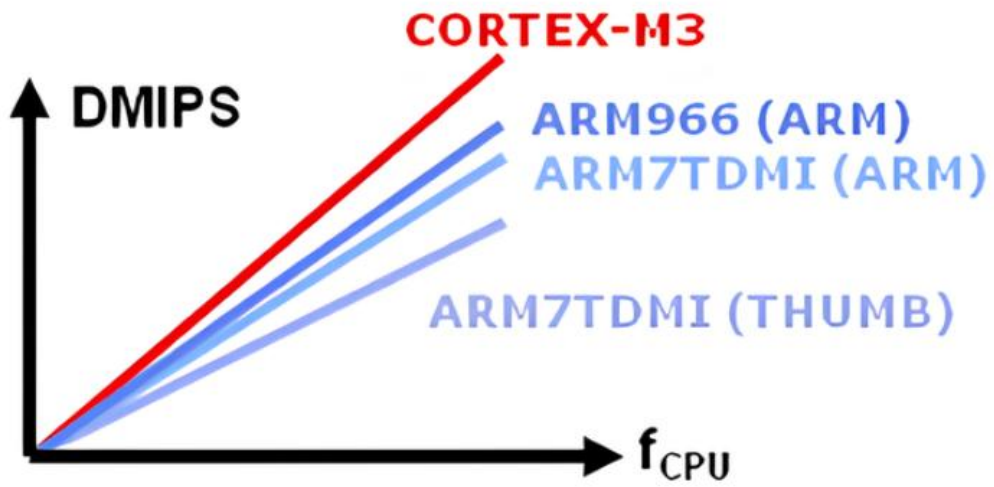
- **ARM**: 32-bit instructions → high performance, but larger code size.
- **Thumb**: 16-bit instructions → smaller code size, but lower performance.

Thumb-2 is a hybrid instruction set that **mixes 16-bit and 32-bit instructions**.

- Keeps the **compactness** of 16-bit instructions.
- Provides the **power** of 32-bit instructions when needed.

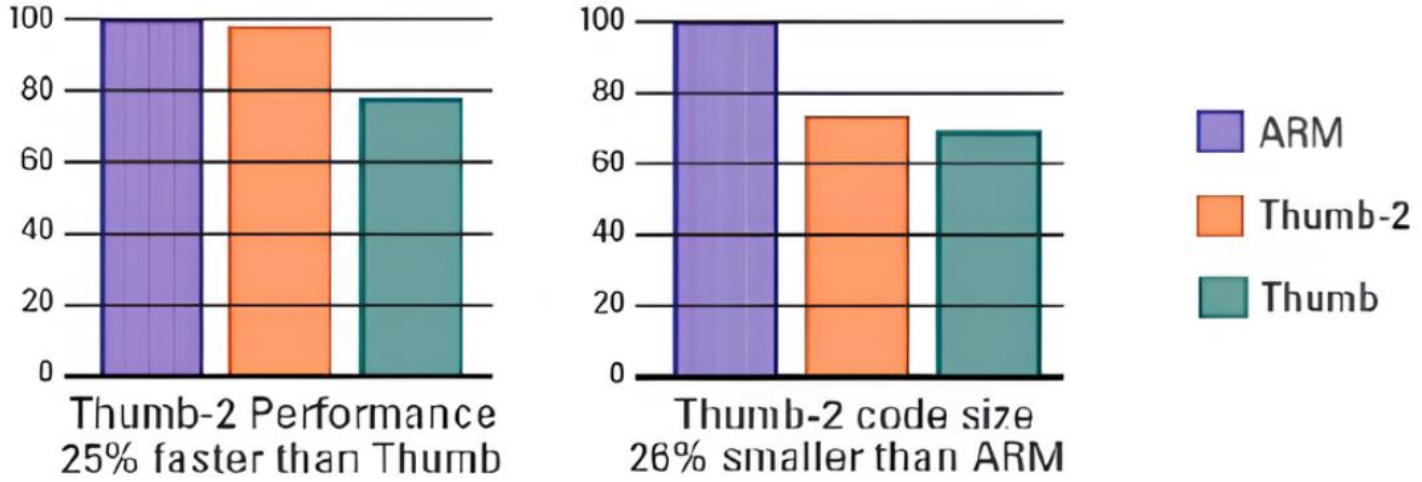
2.2. Core Architecture & Execution

- **Cortex-M3 (red)** → Highest performance due to Thumb-2.
- ARM966 (blue) and ARM7TDMI (light blue) → Lower performance.
- ARM7TDMI (Thumb mode) → Even lower, since Thumb alone sacrifices speed.



- Cortex-M3 with **Thumb-2** delivers the **best performance/efficiency** balance.

2.2. Core Architecture & Execution



Thumb-2 Performance

- Thumb-2 is ~25% **faster** than original Thumb.
- Almost matches pure ARM performance.

Thumb-2 Code Size

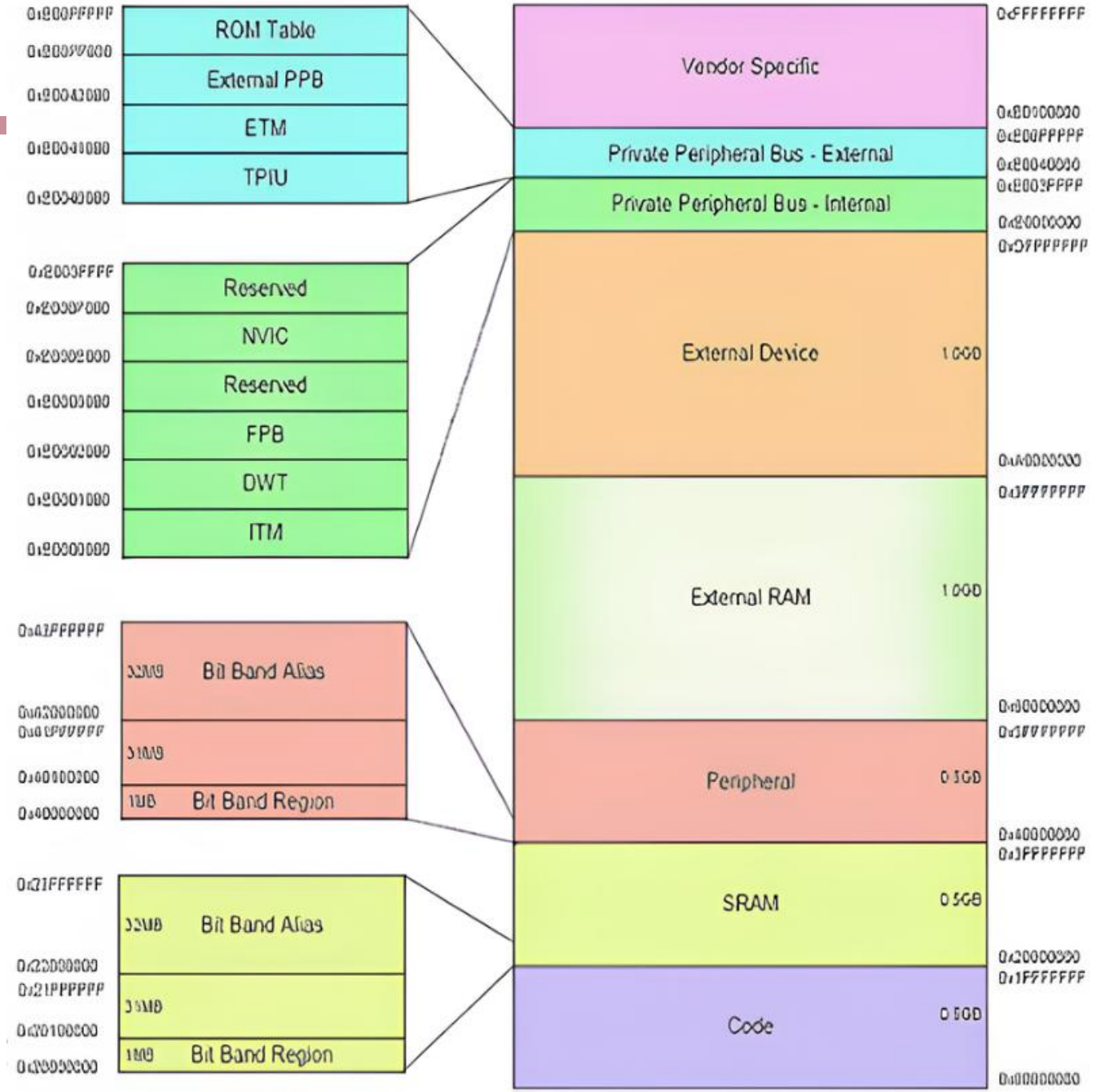
- Thumb-2 code is ~26% **smaller** than ARM code.
- Better for memory-constrained embedded systems.

2.2. Core Architecture & Execution

Memory Mapping

- Cortex-M processors (32-bit) use a **linear 4 GB address space** (0x0000 0000 → 0xFFFF FFFF).

- Different address ranges are reserved for **Code, SRAM, Peripherals, External RAM/Devices, and System control regions**.



2.2. Core Architecture & Execution

- Programmers use **memory-mapped registers** to control hardware.

  Example: Writing to address 0x40021018 might enable a GPIO clock.

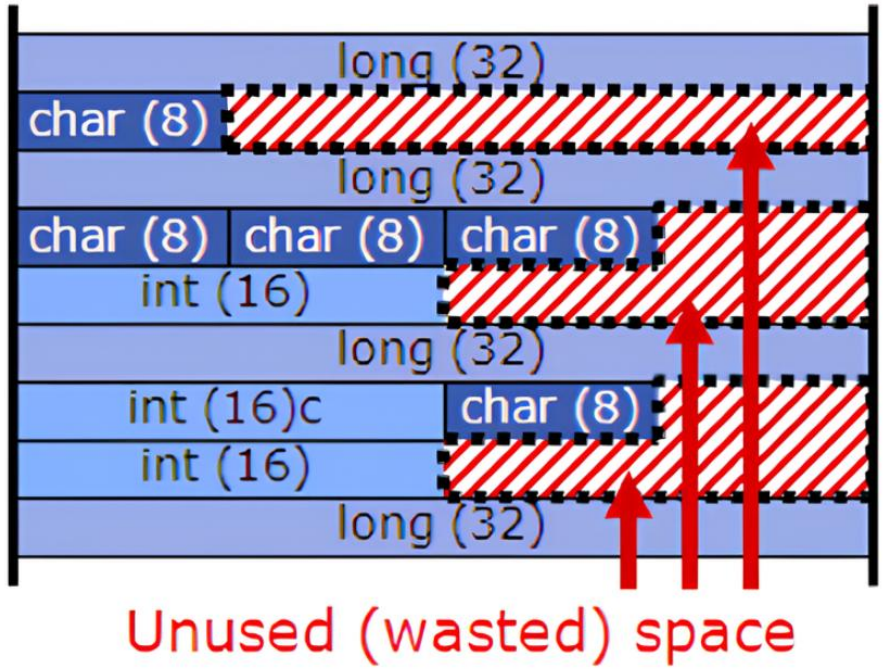
- The memory map ensures **consistent organization** across ARM Cortex-M devices.

- Debugging, stack placement, and linker scripts all depend on Memory map.

2.2. Core Architecture & Execution

Unaligned Memory Access

Processors expect data aligned to boundaries equal to their size.



If variables are placed without alignment:

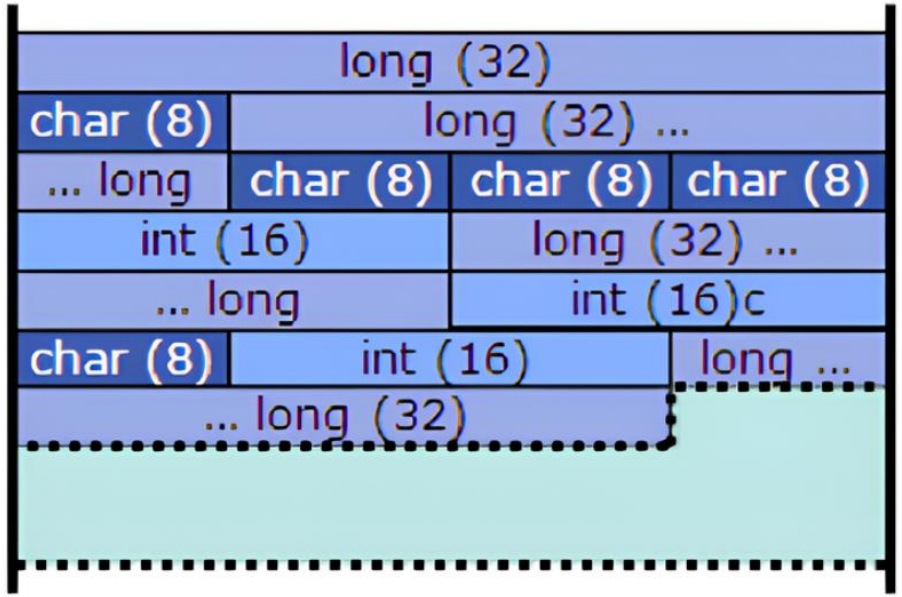
- Some memory bytes remain **unused (wasted space)**, shown in red stripes.
- CPU may need multiple memory cycles to fetch misaligned data → slower execution.

2.2. Core Architecture & Execution

This reduces wasted padding and ensures each data type sits at the correct boundary.

Why It Matters

- In **embedded systems (like STM32)**, memory is limited.
- Poor alignment → wasted RAM + slower CPU performance.
- Good alignment → efficient memory use + faster execution.



Optimized Layout

2.2. Core Architecture & Execution

Code and Data Buses

- Based on **Harvard architecture**:
  - **Separate buses** for instruction fetch (code) and data access.
  - Allows CPU to **fetch instructions and read/write data simultaneously**
  - Faster performance.

2.2. Core Architecture & Execution

System Bus

- Used for **Cortex-M core control and system-level operations**.
- Provides access to the **Cortex Control Space (CCS)**:
  - NVIC (Nested Vectored Interrupt Controller)
  - SysTick timer
  - System control registers

2.2. Core Architecture & Execution

Bus Matrix

- An **interconnect network** that connects all the buses together.
- It **links**:
  - Code/Data buses, System bus, Peripheral bus, External memory interfaces
- Uses **bus arbiters** to handle multiple masters (e.g., CPU, DMA) requesting access at the same time.
- Ensures fair and efficient bus usage.

Contents

1. Overview
   - Intro to STM32 (Applications, Block Diagram, Features)
   - ARM & Cortex Overview
2. Core Architecture & Execution
   - CPU Modes, Switching, Stack Usage
   - Instruction Set (Thumb-2)
   - Memory Map
3. **System Resources**
   - SysTick Timer
   - Interrupts & NVIC
   - CMSIS
4. STM32-Specific Features
   - Clock System (RCC)
   - DMA
5. Software & Tools
   - Driver Libraries
   - Development Workflow
6. Summary

2.3. System Resources

What is the System Timer (SysTick)?

- A **24-bit down counter** inside the Cortex-M core.
- Automatically **reloads** when it reaches zero.
- Can generate an **interrupt** at the end of the count → useful for periodic tasks.

Purpose

- Provides a **regular timer tick** for the **Operating System (OS)**
- Commonly used for:
  - Task scheduling, Time delays, Periodic interrupts for system services.



2.3. System Resources

What is NVIC?

- Nested Vector Interrupt Controller (NVIC)
- NVIC is an **interrupt controller** integrated inside the Cortex-M core.
- It manages **all external and internal interrupts**, ensuring fast and predictable response.

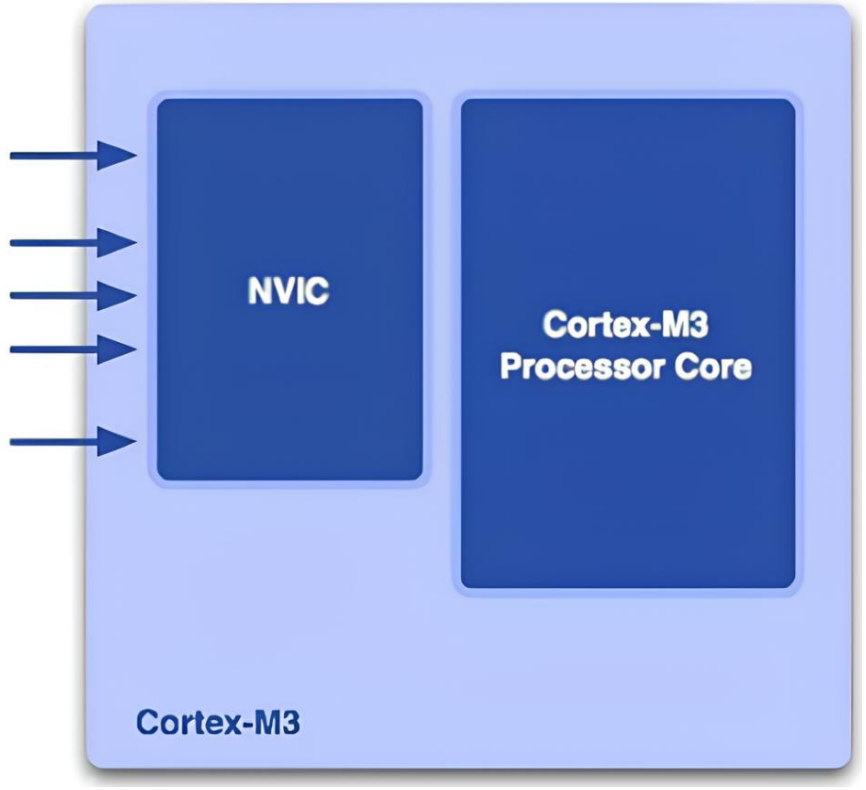


Figure: NVIC inputs.

2.3. System Resources

Why NVIC is Important?

- Enables **real-time responsiveness** in embedded systems.
- Provides **scalability** (different STM32 families support different numbers of interrupts).
- Simplifies **programming** → standard C functions instead of assembly.

2.3. System Resources

Example: NVIC Configuration

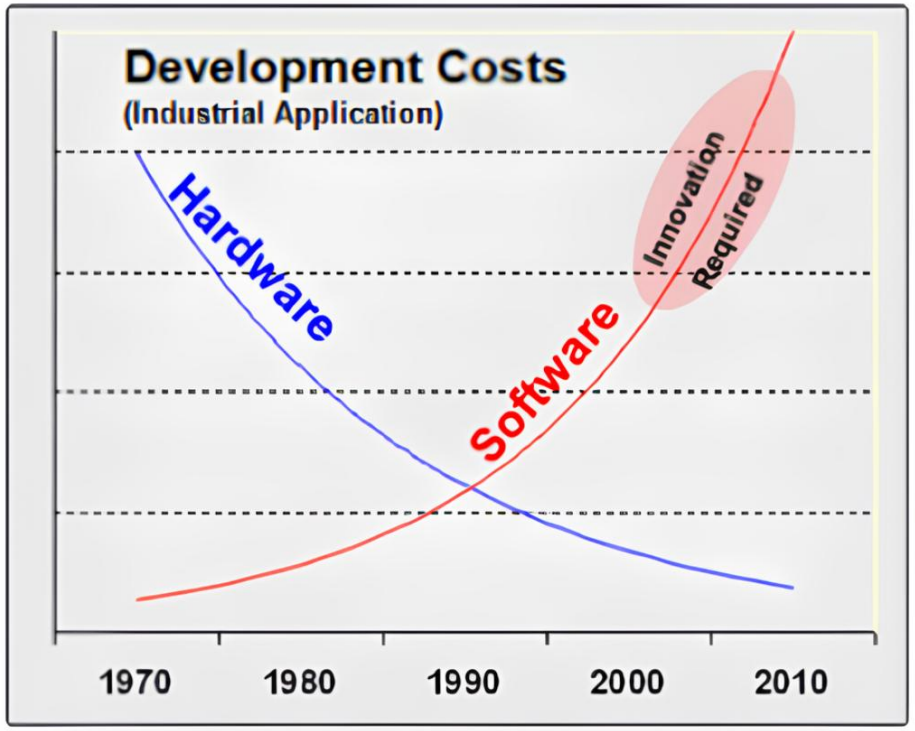
```
HAL_NVIC_SetPriority(EXTI0_IRQn, 2, 0);
HAL_NVIC_EnableIRQ(EXTI0_IRQn);
```

- Enables the **EXTI0 interrupt line** in NVIC.
- Priority = 2 (lower number = higher priority).

2.3. System Resources

Graph shows **hardware vs. software costs** over time:

- In the past (1970s–1980s): **hardware** was expensive, software was cheap.
- (2010s onward): **hardware costs decreased** (cheaper, mass production), but **software costs increased** (more complexity, more features).



- **CMSIS** was introduced to **reduce software development costs** by reusing code across platforms.

2.3. System Resources

What is CMSIS?

- Cortex Microcontroller Software Interface (CMSIS)
- A **software standard** defined by ARM for all Cortex-M microcontrollers.
- Provides a **vendor-independent abstraction layer.**
- A **code** written for one ARM Cortex-M MCU can be reused on another without major changes.
- Designed to **simplify** development, portability, and reuse of software across different vendors (e.g., STMicroelectronics STM32, NXP, TI, etc.).

2.3. System Resources

Key Features:

- **Core Access Functions** → Easy access to NVIC, SysTick, and system registers.
- **Device Peripheral Access** → Standard naming for registers (GPIOA->ODR, USART1->DR).
- **DSP Library** → Optimized math functions (FFT, filters, vector ops).
- **RTOS API** → Standardized interface for real-time operating systems.

2.3. System Resources

Example: CMSIS

```
NVIC_EnableIRQ(EXTI0_IRQn);
```

- This code works across STM32, NXP, TI ARM chips.

2.3. System Resources

Flow of Execution

- **Application code** calls RTOS services or middleware.
- Middleware & RTOS call **CMSIS functions**.
- CMSIS interacts directly with the **MCU hardware** (CPU, SysTick, NVIC, peripherals).

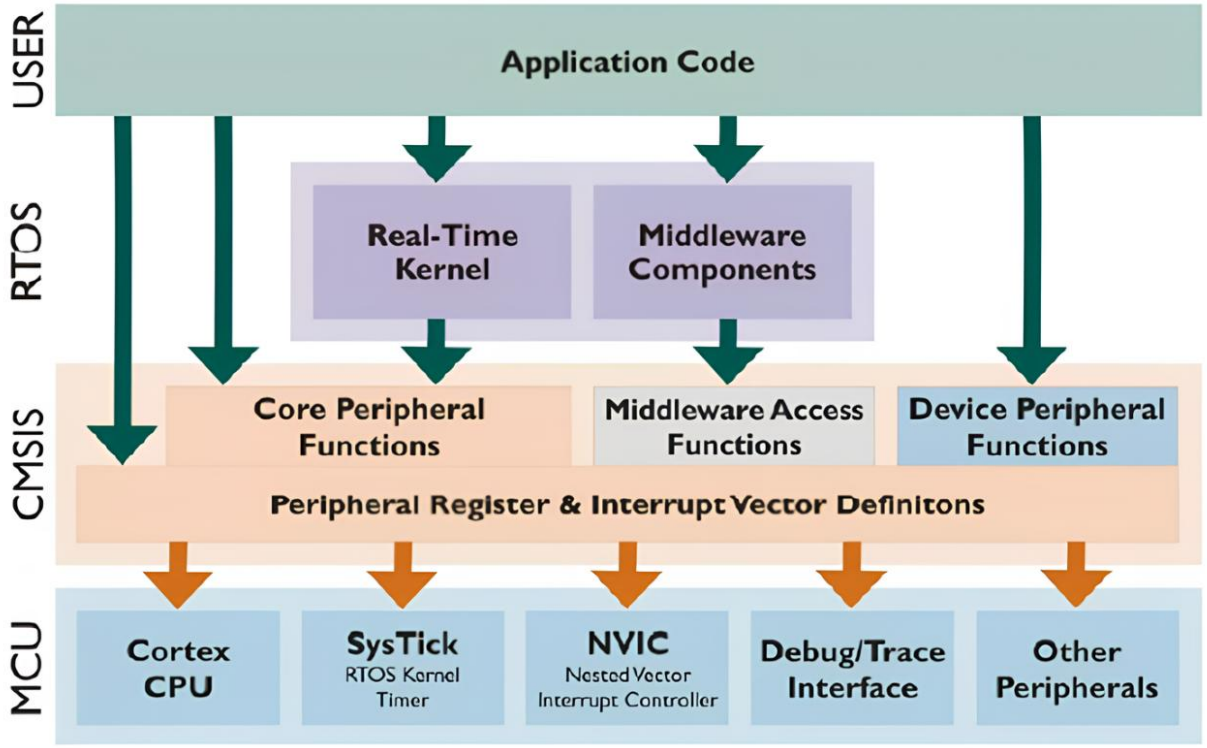


Figure: CMSIS Structure with Layers.

Contents

1. Overview
   - Intro to STM32 (Applications, Block Diagram, Features)
   - ARM & Cortex Overview
2. Core Architecture & Execution
   - CPU Modes, Switching, Stack Usage
   - Instruction Set (Thumb-2)
   - Memory Map
3. System Resources
   - SysTick Timer
   - Interrupts & NVIC
   - CMSIS
4. **STM32-Specific Features**
   - Clock System (RCC)
   - DMA
5. Software & Tools
   - Driver Libraries
   - Development Workflow
6. Summary

2.4. STM32-Specific Features

Clock Schemes in STM32

Oscillators

STM32 has **four clock sources**:

External Oscillators:

- **HSE (**High-Speed External Oscillator**)** → 4–16 MHz crystal/clock input.
- **LSE (**Low-Speed External Oscillator**)** → 32.768 kHz crystal (for RTC).

Internal Oscillators:

- **HSI (**High-Speed Internal RC oscillator**)** → 8 MHz factory-trimmed RC oscillator.
- **LSI (**Low-Speed Internal RC oscillator**)** → ~32 kHz, low accuracy (used for watchdog, backup).

2.4. STM32-Specific Features

PLL (Phase-Locked Loop)

- The PLL can **multiply the frequency** of HSE or HSI.
- Configurable from ×2 up to ×16.
- Provides **SYSCLK** up to **72 MHz**.
- This is usually the **main clock source** for the MCU.
- Different buses (AHB, APB1, APB2) ensure optimized speed/power for each block.

This flexible scheme allows STM32 to balance **performance vs. power consumption** depending on application needs.

2.4. STM32-Specific Features

Clocking Configurations in STM32

Reset & Clock Control (RCC):



- It chooses where the clock comes from.
- It adjusts the speed.
- It only powers up the parts you need.

This keeps the microcontroller **flexible and energy-efficient**

2.4. STM32-Specific Features

Control Registers:

Special registers control clock features:

- **Enable/Disable** internal (HSI) or external (HSE) oscillators.
- **PLL settings** to multiply/divide the clock.
- **Prescalers** to slow down the clock for different buses (AHB, APB, ADC, USB).
- **Status flags** show if an oscillator is ready or not.

2.4. STM32-Specific Features

Bus Control:

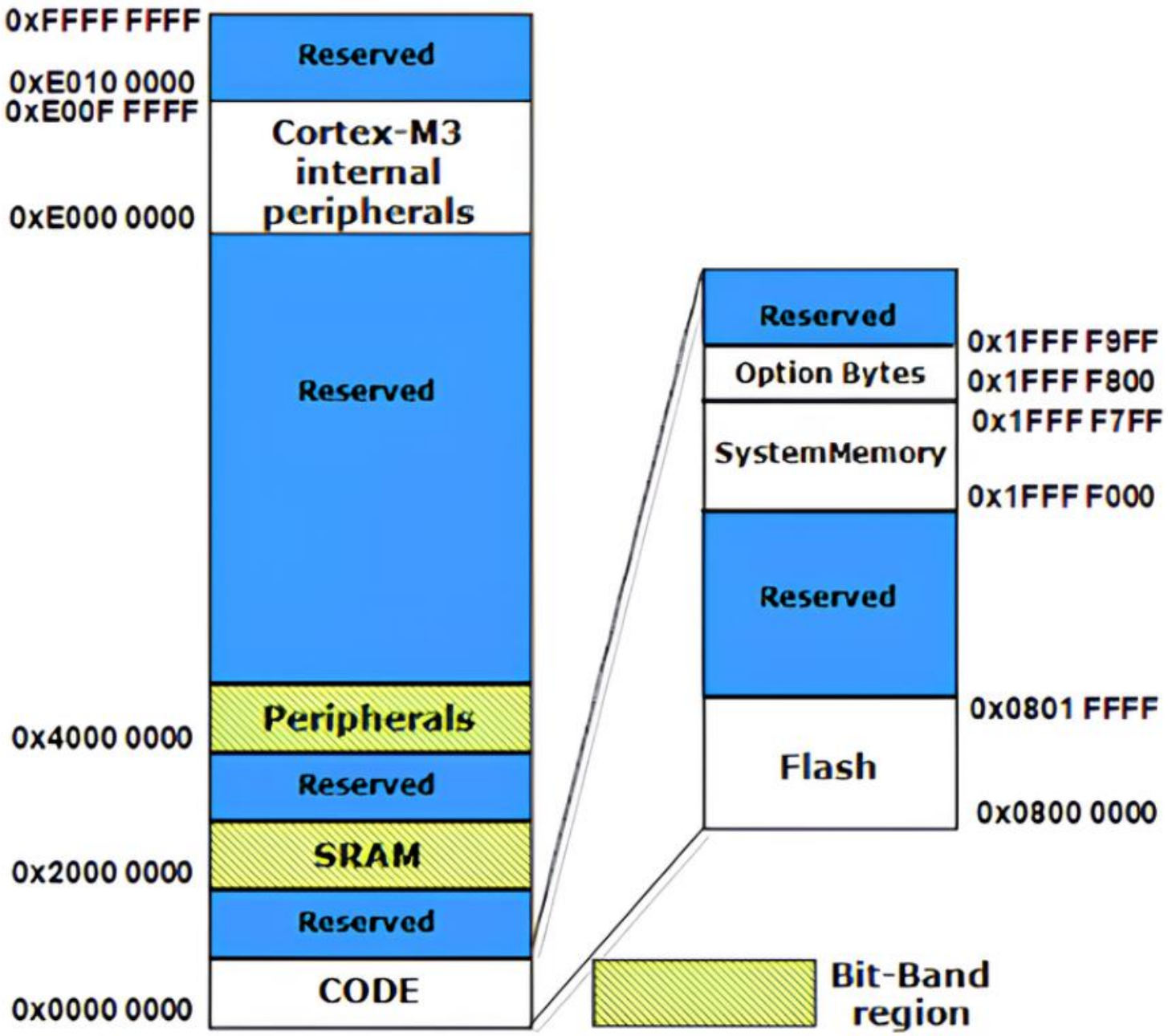
RCC can **reset or enable/disable clocks** to different buses:

- **APB2** → High-speed peripherals (like GPIO, ADC, USART1).
- **APB1** → Low-speed peripherals (like USART2–5, I²C, CAN, timers).
- **AHB** → DMA, memory interface, and other high-performance blocks.

2.4. STM32-Specific Features

Memory map organizes Flash, SRAM, peripherals, and Cortex-M3 core registers in a linear 4 GB space.

- The Cortex-M3 has a **4 GB address space (0x0000 0000 → 0xFFFF FFFF)** divided into regions.



2.4. STM32-Specific Features

Boot mode pins (BOOT0, BOOT1) decide where the MCU starts execution after reset:

- Flash (user code).
- System memory (bootloader).
- SRAM (debugging/testing).

BOOT Mode Selection Pins		Boot Mode	Aliasing
BOOT1	BOOT0		
x	0	User Flash	User Flash is selected as boot space
0	1	SystemMemory	SystemMemory is selected as boot space
1	1	Embedded SRAM	Embedded SRAM is selected as boot space

- This mechanism allows STM32 to be **flexible**: run user applications, enter programming mode, or debug mode.

2.4. STM32-Specific Features

Direct Memory Access (DMA) in STM32

Independent Channels

STM32 has **7 independent, configurable DMA channels**.

- Each channel can be triggered by **hardware requests** (e.g., ADC ready, USART transmit)

  or **software requests**.

2.4. STM32-Specific Features

Transfer Types

- **Memory-to-Memory** → Copy data blocks directly between two memory areas.
- **Memory-to-Peripheral** → Send data (e.g., buffer → UART, SPI).
- **Peripheral-to-Memory** → Receive data (e.g., ADC → buffer in RAM).

This frees the CPU from repeatedly copying data.

2.4. STM32-Specific Features

Priorities

- **Software-programmable priorities:** The developer can assign priorities to channels.
- **Hardware arbitration:** If two channels have the same priority, the hardware decides which goes first.

2.4. STM32-Specific Features

Programmable & Independent Transfer

- Each channel can be customized to transfer:
  - **Byte (8-bit), half-word (16-bit), or word (32-bit)**.
  - **Source and destination** are independent (e.g., peripheral ↔ memory).

This allows flexible data handling for different applications.

2.4. STM32-Specific Features

DMA Channel Configuration

Each DMA channel requires configuration:

- **Number of Data:** How many items to transfer.
- **Peripheral Address:** Source/destination if using a peripheral.
- **Memory Address:** Source/destination if using memory.

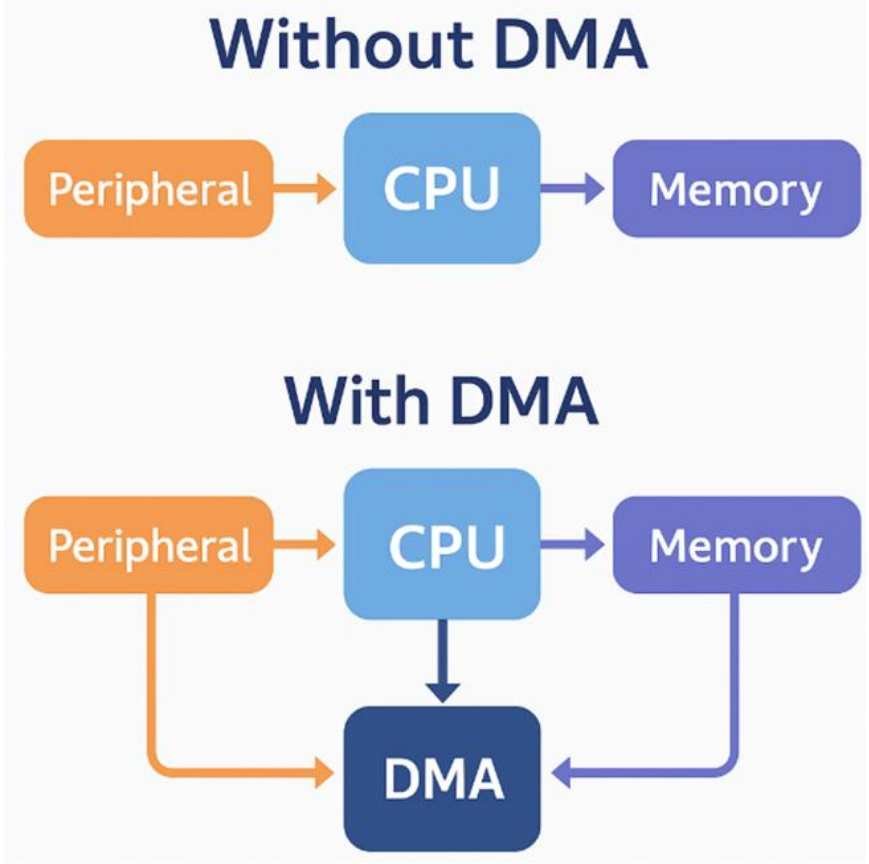


Figure: DMA Processing.

2.4. STM32-Specific Features

Control of Each DMA Channel

Each DMA channel is managed using:

- **4 registers:** configure source, destination, number of transfers, and settings.
- **3 interrupts** that notify the CPU when:
  - **Transfer complete** (all data moved).
  - **Half transfer** (useful for continuous streaming, e.g., ADC sampling).
  - **Error** (e.g., wrong address, bus fault).

This mapping allows STM32 to efficiently handle **parallel data transfers** from multiple peripherals without CPU overhead.

Contents

1. Overview
   - Intro to STM32 (Applications, Block Diagram, Features)
   - ARM & Cortex Overview
2. Core Architecture & Execution
   - CPU Modes, Switching, Stack Usage
   - Instruction Set (Thumb-2)
   - Memory Map
3. System Resources
   - SysTick Timer
   - Interrupts & NVIC
   - CMSIS
4. STM32-Specific Features
   - Clock System (RCC)
   - DMA
5. **Software & Tools**
   - Driver Libraries
   - Development Workflow
6. Summary

2.5. Software & Tools

STM32 Driver Libraries

Standard Peripheral Library (SPL)

- **Older, low-level library** from ST.
- Functions are close to the registers → more control, but more verbose.
- Often used in STM32F1 series (e.g., STM32F103).
- Not maintained for new families anymore.

2.5. Software & Tools

Example: Toggle an LED on PC13 using SPL.

```
// Enable clock for GPIOC
RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOC, ENABLE);

GPIO_InitTypeDef GPIO_InitStructure;
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_13;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_2MHz;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP; // Push-pull output
GPIO_Init(GPIOC, &GPIO_InitStructure);

while (1)
{
    GPIO_SetBits(GPIOC, GPIO_Pin_13);   // LED OFF (on some boards)
    for (int i=0; i<500000; i++);       // Delay
    GPIO_ResetBits(GPIOC, GPIO_Pin_13); // LED ON
    for (int i=0; i<500000; i++);       // Delay
}
```

2.5. Software & Tools

HAL (Hardware Abstraction Layer)

- **Modern high-level library** provided in STM32Cube.
  - Easier to use, portable across STM32 families.
  - Uses initialization structures and HAL drivers.
  - Some performance overhead compared to SPL, but simpler and well-supported.

2.5. Software & Tools

Example: Toggle an LED on PC13 using HAL.

```
HAL_Init();                    // Initialize HAL
__HAL_RCC_GPIOC_CLK_ENABLE(); // Enable clock for GPIOC

GPIO_InitTypeDef GPIO_InitStruct = {0};
GPIO_InitStruct.Pin = GPIO_PIN_13;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

while (1)
{
    HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13); // Toggle LED
    HAL_Delay(500);                         // Delay 500 ms
}
```

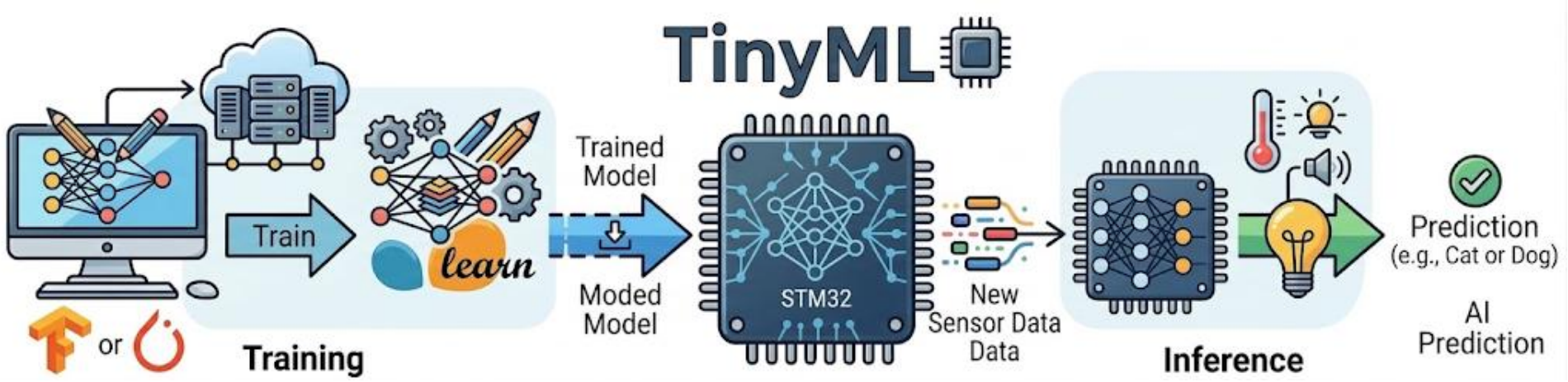
2.5. Software & Tools

Key Differences in Syntax

Feature	SPL Example	HAL Example
Clock enable	RCC_APB2PeriphClockCmd(...)	__HAL_RCC_GPIOC_CLK_ENABLE()
GPIO init	GPIO_Init(GPIOC, &GPIO_InitStructure);	HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);
LED control	GPIO_SetBits() / GPIO_ResetBits()	HAL_GPIO_TogglePin()
Delay	Manual loop (blocking)	HAL_Delay() (SysTick-based)

- **SPL**: More control, but verbose. Best if you want direct register-level behavior.
- **HAL**: Easier and portable. Best for modern projects with STM32CubeIDE.

2.5. Software & Tools



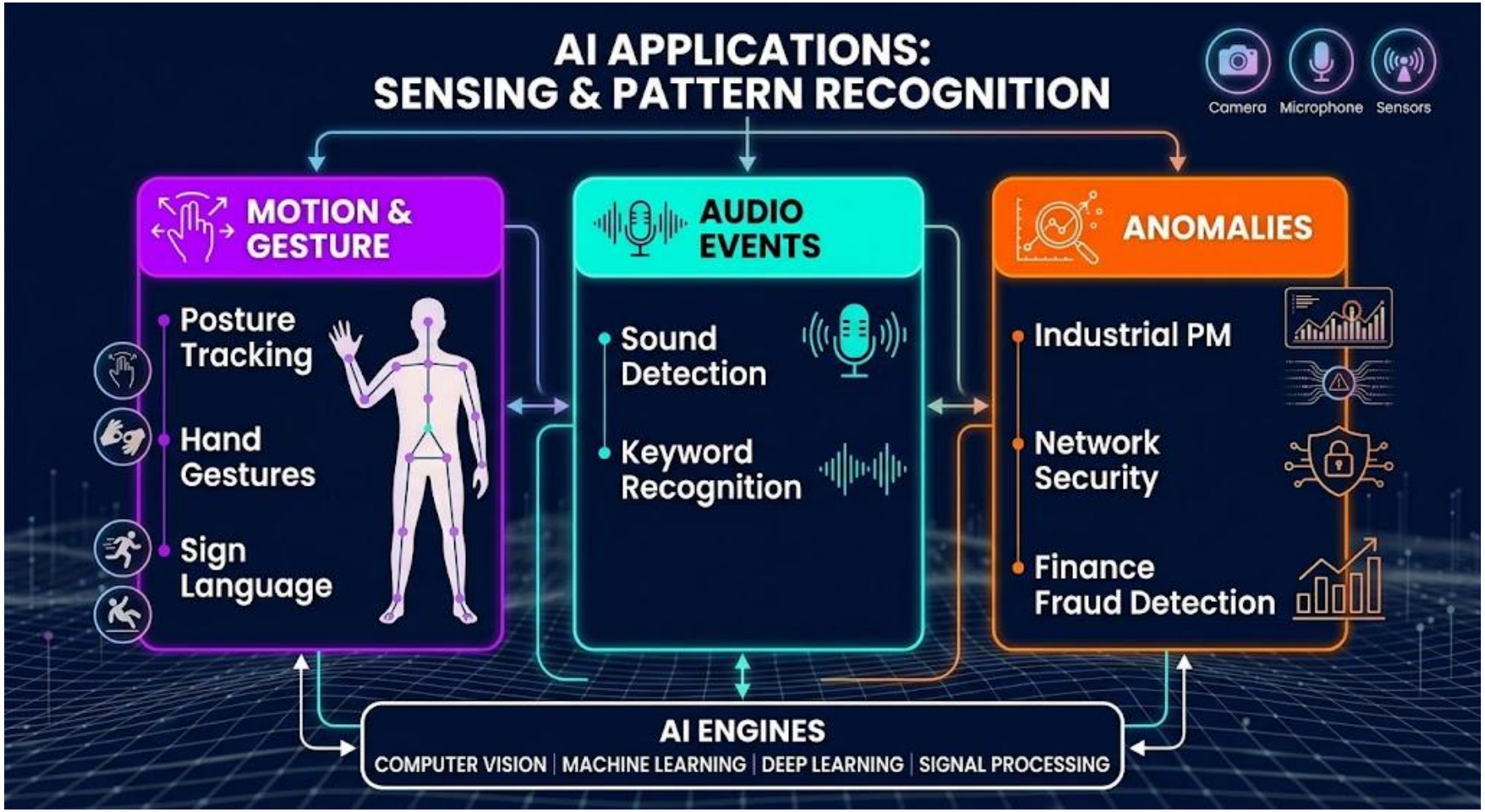
- STM32 can run small **AI** and **Machine learning** models (Cortex M4 and above)

Training: PC or cloud using frameworks like TensorFlow or PyTorch.

Inference: The STM32 only runs the already-trained model to make predictions on new sensor data.

2.5. Software & Tools

STM32 is well-suited for tasks with limited, structured input data and clear outputs:



2.5. Software & Tools

IDE Options

- **MDK-ARM (Keil)**
  - Commercial IDE, very popular in industry.
  - Great debugger and optimization tools.
  - Free version limited to **32 KB code size**.
  - Works well with STM32, supports **CMSIS and SPL/HAL**.

- **STM32CubeIDE (by STMicroelectronics)**
  - **Free and official** IDE for STM32.
  - Based on Eclipse + GCC + CubeMX configurator.
  - Integrated support for **HAL drivers, LL drivers, FreeRTOS, and CubeMX GUI**.
  - Best choice for students and beginners.

Contents

1. Overview
   - Intro to STM32 (Applications, Block Diagram, Features)
   - ARM & Cortex Overview
2. Core Architecture & Execution
   - CPU Modes, Switching, Stack Usage
   - Instruction Set (Thumb-2)
   - Memory Map
3. System Resources
   - SysTick Timer
   - Interrupts & NVIC
   - CMSIS
4. STM32-Specific Features
   - Clock System (RCC)
   - DMA
5. Software & Tools
   - Driver Libraries
   - Development Workflow
6. **Summary**

2.6. Summary

- **STM32** is a versatile ARM-based MCUs.
- **Core architecture**: Cortex-M3, NVIC, SysTick.
- **Memory**: Flash, SRAM, registers.
- **Tools**: CMSIS, HAL, RCC.
- **IDE workflow**: Keil, STM32CubeIDE.

- **Thread Mode** = runs application code (can be privileged or not).
- **Handler Mode** = runs interrupts/exceptions (always privileged).
- **Two Stacks** = Main stack (for OS/interrupts), Process stack (for user code).
- **Privilege Levels** = Protect the system: OS runs privileged, apps can run unprivileged.

2.6. Summary

- Cortex-M has **16 registers (R0–R15) + xPSR**.
- **SP, LR, PC** are special purpose → for stack, function return, program flow.
- **xPSR** stores status flags, execution state, and interrupt info.
- **Pipeline (Fetch–Decode–Execute)** increases speed by overlapping instructions.

- SysTick = **24-bit down counter** in Cortex-M.
- Provides **OS timer ticks** for scheduling.
- Has **3 registers**: Control & Status, Reload Value, Current Value.
- Used for **periodic interrupts, delays, and task switching** in embedded systems.

